NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPPALLI

Department of Computer Science and Engineering

RouteCraft

Multi-Depot Vehicle Routing Studio

Data Structures Laboratory Project Report
Course: CSLR31

Team AlgoSmiths

Meesala Hariteja	106125071
Mayank Gupta	106125069
Matta Jay Paul	106125067

Academic Year 2026

Abstract

RouteCraft is a desktop application developed to demonstrate the use of data structures and algorithms in a practical logistics problem: the capacitated multi-depot vehicle routing problem. A delivery organization must serve customers with different demands using vehicles stationed at multiple depots, while respecting vehicle capacities and road-network restrictions. RouteCraft provides an interactive Java Swing interface and a native C++17 routing engine connected through the Java Native Interface (JNI). The system supports customer and depot configuration, fleet setup, OpenStreetMap (OSM) road-network import, route construction, route evaluation, visualization and dispatch simulation. The road network is represented as a directed weighted graph. Dijkstra's algorithm is used for shortest-path calculations, while Kosaraju's algorithm identifies strongly connected components and the implementation retains the largest component. Customer assignment uses a capacity-aware distance heuristic, followed by depot-wise route planning and customer reassignment when a depot cannot serve its assigned demand with its configured fleet. Routes are constructed using Nearest Neighbor or Clarke–Wright Savings and refined using 2-opt. The project illustrates how graph algorithms, greedy heuristics, local search and data structures can be integrated into a working software system. It is an educational heuristic application and does not guarantee a globally optimal solution.

Contents

Abstract **i**

1 Step 1 – Problem Identification **1**
1.1 Problem Statement . . . 1
1.2 Motivation . . . 1
1.3 Objectives . . . 1
1.4 Existing Approach and Project Scope . . . 2
1.5 Salient Features . . . 2
1.6 Social Relevance . . . 2

2 Step 2 – Product Design **3**
2.1 System Architecture . . . 3
2.2 System Flowchart . . . 3
2.3 Module Description and Working . . . 3
2.3.1 User Interface and Scenario Management . . . 3
2.3.2 Road-Network Processing . . . 5
2.3.3 Customer Clustering and Depot Assignment . . . 5
2.3.4 Route Construction . . . 5
2.3.5 Fleet Allocation and Shortage Handling . . . 5
2.3.6 Route Evaluation and Improvement . . . 6
2.3.7 Java–C++ Integration . . . 6
2.4 End-to-End Operation . . . 6

3 Step 3 – Coding and Modular Organization **7**
3.1 Implementation Technologies . . . 7
3.2 Directory Organization . . . 7
3.3 Important Data Structures . . . 8
3.4 Algorithms Used . . . 8
3.5 Sample Code: 2-opt Improvement . . . 8
3.6 Illustrative Algorithm: 2-opt . . . 9

4 Step 4 – Testing and Test Data **10**
4.1 Testing Approach . . . 10
4.2 Native Test Cases . . . 10
4.3 Representative Test Data . . . 11
4.4 Testing Limitations . . . 11

5 Step 5 – Contributions, Key Takeaways and Future Scope **12**
5.1 Team Contributions 12
5.2 Key Takeaways 12
5.3 Future Scope 13
5.4 Acknowledgements 13
5.5 References 13

Chapter 1

Step 1 – Problem Identification

1.1 Problem Statement

Imagine a delivery company operating from multiple warehouses across a city. Customers are spread across different locations, and each customer has a specific quantity of goods to receive. The company must decide which depot should serve each customer, how vehicles should be assigned, and in what order the customers should be visited. This is not simply a shortest-path problem: vehicles have limited carrying capacities, depots have limited fleets, and routes must follow the available road network. Poor planning can cause unnecessary travel, inefficient fleet utilization, overloaded vehicles or customers remaining unserved.

The objective of RouteCraft is to provide an interactive system that accepts depot, customer, demand and fleet information, optionally uses real road-network data, and constructs capacity-aware delivery routes for multiple depots. It also evaluates route quality and displays the resulting plan so that users can inspect and simulate dispatch.

1.2 Motivation

Last-mile delivery and distribution planning involve several related decisions. A shortest-path algorithm can find a path between two locations, but it cannot by itself decide which depot should serve a customer, how customers should be grouped, how many vehicles are needed, or how vehicle capacity should be respected. RouteCraft was selected because it brings these decisions together and provides a practical setting for applying graph representations, greedy algorithms, hashing, stacks, priority queues and local search.

1.3 Objectives

1. Model road networks as directed, weighted graphs.
2. Allow users to create multiple depots and customers with individual delivery demands.
3. Configure vehicle counts and carrying capacities for each depot.
4. Assign customers to depots while considering distance and assigned fleet capacity.
5. Construct feasible routes using Nearest Neighbor or Clarke–Wright Savings.

6. Handle fleet shortages using dedicated, local ad-hoc and virtual planning tiers, and attempt customer reassignment where another depot has capacity.
7. Evaluate routes and improve visit order using 2-opt.
8. Display routes and support inspection, dispatch simulation, scenario handling and export.

1.4 Existing Approach and Project Scope

A basic navigation system generally focuses on finding a path between a specified origin and destination. A vehicle-routing system must solve a broader problem involving multiple customers, depots, vehicle capacities and fleet availability. RouteCraft is an educational prototype that demonstrates selected parts of this broader problem. It uses heuristic methods to generate practical candidate plans; it does not claim to find a globally optimal MDVRP solution. Travel time and operating cost are estimates based on the application's road and scoring model, not live traffic data or commercial quotations.

1.5 Salient Features

- Java Swing desktop interface for scenario creation and route visualization.
- Multiple depots, customers, delivery demands and depot-specific fleet configuration.
- Optional import of OpenStreetMap XML road data.
- Capacity-aware initial customer-to-depot assignment and customer reassignment.
- Nearest Neighbor and Clarke–Wright Savings route construction.
- Dedicated, local ad-hoc and virtual planning fleet tiers.
- Dijkstra shortest paths, Kosaraju strongly connected components and 2-opt route improvement.
- Route metrics, map visualization, dispatch simulation, scenario save/open and route export.

1.6 Social Relevance

More systematic delivery planning can support better utilization of vehicles and urban road infrastructure. The project is relevant to UN Sustainable Development Goal 11 (Sustainable Cities and Communities), through its focus on urban distribution planning. The prototype does not claim a measured reduction in emissions, fuel use or delivery cost.

Chapter 2

Step 2 – Product Design

2.1 System Architecture

RouteCraft is organized into four cooperating layers:

1. **Presentation layer:** Java Swing provides the user interface, input controls, map display, route visualization, dispatch simulation and export features.
2. **Java optimization coordinator:** manages scenario data, initial customer clustering, depot-wise planning, fleet allocation, customer reassignment and comparison of alternative plans.
3. **JNI adapter:** transfers graph and optimization data between Java and C++ and returns native results to the Java application.
4. **C++17 DSA engine:** implements graph processing, route construction, route evaluation, route improvement and shared data models.

2.2 System Flowchart

Figure 2.1 gives a complete view of RouteCraft from user input to the final delivery plan. It is intentionally organized into stages so that a reader can follow both the normal route-planning path and the decisions made when a depot does not have enough vehicles. The diagram also distinguishes customer reassignment from vehicle movement: customers may be reassigned to another depot, but vehicles remain based at their home depots.

2.3 Module Description and Working

2.3.1 User Interface and Scenario Management

The main interface is implemented in `RouteCraftVRP.java`. It allows users to add, edit and remove depots and customers, enter coordinates and customer demand, configure vehicle counts and capacities, and choose optimization settings. The map area presents the road network, depots, customers and generated routes. The interface also provides controls for route inspection, dispatch simulation, pause/resume, scenario save/open, undo/redo where

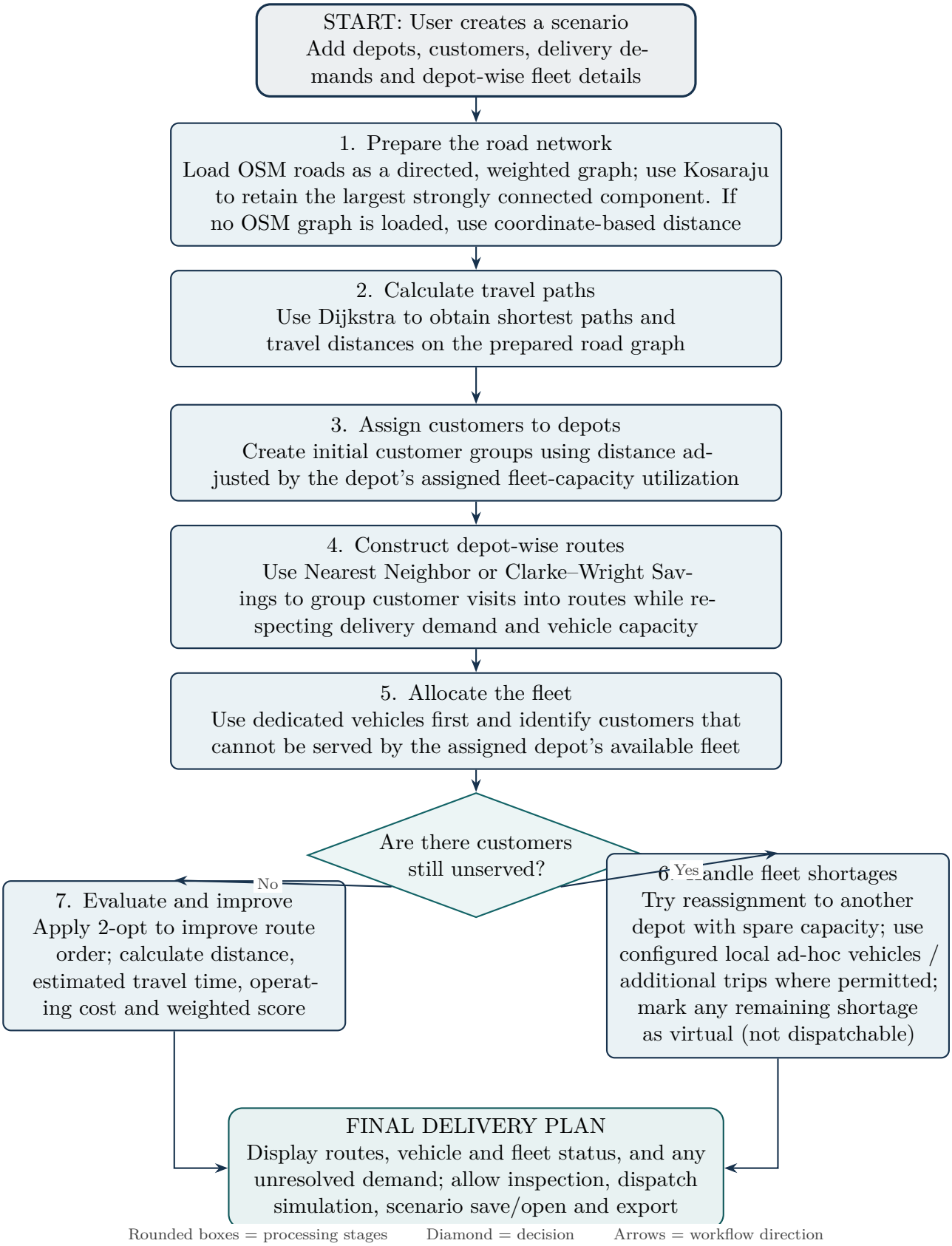


Figure 2.1: RouteCraft end-to-end workflow, from user inputs and road preparation through customer assignment, route construction, fleet handling and final route output.

supported, and route export. The interface is designed to let users build a scenario without directly interacting with the underlying algorithms.

2.3.2 Road-Network Processing

When an OSM extract is loaded, the road-network module reads supported map nodes and road ways and creates a directed weighted graph. An adjacency list stores outgoing road edges. Two-way roads are represented by two directed edges, while one-way roads are represented in the permitted direction. Kosaraju's algorithm identifies strongly connected components; the implementation retains the largest component in the road-node set and removes edges that lead outside it. Dijkstra's algorithm is then used to calculate shortest paths through the retained graph. If a road graph is not loaded, the application can use coordinate-based geographic distance calculations as a fallback.

2.3.3 Customer Clustering and Depot Assignment

The Java coordinator initially assigns each customer to a depot using a distance score adjusted by the current assigned-demand utilization of that depot. This makes a nearby depot less attractive as its assigned demand approaches its configured capacity. The method is a greedy heuristic: it discourages overloading but does not guarantee globally optimal clusters.

After initial assignment, RouteCraft builds a plan for each depot. If customers remain on virtual routes, the coordinator attempts to move selected customers to another depot only when that receiving depot can serve the resulting cluster with its own configured fleet. Vehicles themselves do not move between depots; only customer assignments change. The affected depot plans are recalculated after a reassignment.

2.3.4 Route Construction

Route construction is performed by the C++ DSA engine. In Nearest Neighbor, the algorithm repeatedly selects a nearby unserved customer that fits in the remaining vehicle capacity. When no additional customer can be added, the route is closed and another vehicle route is started if fleet capacity permits. In Clarke–Wright Savings, the algorithm begins with individual customer routes, calculates savings for joining customer pairs, sorts candidate savings, and merges compatible route endpoints while checking capacity and route constraints.

2.3.5 Fleet Allocation and Shortage Handling

The system distinguishes three fleet tiers. Dedicated vehicles are the primary vehicles configured at a depot. Local ad-hoc vehicles can be activated up to the configured local maximum. Virtual routes are planning-only indicators for demand that cannot be covered by the available real fleet; they are not dispatchable vehicles. The planning process attempts dedicated fleet allocation first, then local ad-hoc activation, and customer reassignment to other depots where feasible. A remaining virtual route communicates that the configured fleet is insufficient for all demand.

2.3.6 Route Evaluation and Improvement

The route-improvement module calculates route distance, estimated travel time, estimated operating cost and a weighted score. The 2-opt heuristic examines pairs of route edges and reverses a subsequence when the resulting route has a better score. It continues until no improving change is found or the implementation's iteration bound is reached. These heuristics improve a candidate plan but do not prove global optimality.

2.3.7 Java–C++ Integration

The Java coordinator communicates with the native engine through JNI. The JNI adapter converts Java scenario information into native representations, invokes the C++ functions and transfers route results and metrics back to Java. This separation keeps the interface and the computational engine modular. It also introduced integration work: data types, identifiers, memory ownership and error handling must remain consistent across the Java and C++ layers.

2.4 End-to-End Operation

A user creates depots and customers, enters delivery demand and configures each depot's fleet. The user may load an OSM road extract to enable road-aware routing. On optimization, the Java coordinator creates initial customer clusters and coordinates depot-level planning. The C++ engine constructs routes and computes route metrics, while the Java layer handles fleet planning and customer reassignment. The selected plan is returned to the interface, where routes and fleet status can be inspected, simulated and exported.

Chapter 3

Step 3 – Coding and Modular Organization

3.1 Implementation Technologies

- **Java:** Swing interface and optimization coordination.
- **C++17:** native graph, route-construction and route-improvement algorithms.
- **JNI:** communication between the Java application and native C++ engine.
- **OpenStreetMap XML:** optional source of real road-network geometry and direction information.
- **Makefile and Windows launcher:** build and launch support for the configured development environment.

3.2 Directory Organization

```
RouteCraft/
|-- Source/
|   |-- DSA/
|   |   |-- RoadNetworkAlgorithms.cpp
|   |   |-- RouteConstructionAlgorithms.cpp
|   |   |-- RouteImprovementAlgorithms.cpp
|   |   |-- RouteObjects.cpp
|   |   `-- RouteCraftDsaTests.cpp
|   |-- Java/
|   |   |-- RouteCraftVRP.java
|   |   |-- RouteOptimizerBridge.java
|   |   `-- RouteOptimizerBridgeTest.java
|   `-- JNI/RouteOptimizerJni.cpp
|-- include/
|   |-- DSA/
|   `-- JNI/
|-- Document/
```

```
|-- Makefile
`-- run.bat
```

3.3 Important Data Structures

Data structure	Role in RouteCraft
Adjacency list	Stores outgoing edges for each road node in the directed road graph.
Priority queue	Selects the next minimum-distance candidate efficiently in Dijkstra's algorithm.
Stack	Supports finishing-order processing in Kosaraju's two-pass depth-first search.
Hash maps	Provide lookup by node identifiers and support caches and grouped scenario data.
Vectors / dynamic arrays	Store customers, depots, route stops, edges and candidate results.
Sets	Help maintain unique graph-node collections and retained SCC membership.

3.4 Algorithms Used

Algorithm	Application
Dijkstra's shortest path	Finds a minimum-weight path through the loaded directed road graph.
Kosaraju's SCC algorithm	Identifies strongly connected components; the largest component is retained during road-graph preparation.
Capacity-aware greedy assignment	Assigns customers to depots using distance adjusted by current assigned-demand utilization.
Nearest Neighbor	Builds a route by repeatedly selecting a nearby feasible unserved customer.
Clarke–Wright Savings	Merges route endpoints based on pairwise savings, subject to feasibility and capacity checks.
Customer reassignment	Attempts to move customers from virtual routes to another depot whose own fleet can serve the resulting cluster.
2-opt local search	Reverses route subsequences when the route score improves.

3.5 Sample Code: 2-opt Improvement

The following excerpt illustrates the route-improvement logic in the C++ implementation. It compares the two boundary edges before and after a proposed reversal and accepts the reversal when it reduces the route distance. The implementation uses an iteration limit as a safety bound.

Listing 3.1: Representative 2-opt improvement loop from RouteImprovementAlgorithms.cpp

```
if (distance(before, last) + distance(first, after) +
        TWO_OPT_EPSILON <
    distance(before, first) + distance(last, after)) {
    for (size_t left = i, right = k; left < right;
         ++left, --right) {
        auto value = std::move(route.stops[left]);
        route.stops[left] = std::move(route.stops[right]);
        route.stops[right] = std::move(value);
    }
    improved = true;
}
```

3.6 Illustrative Algorithm: 2-opt

For a route containing an ordered sequence of stops, 2-opt considers replacing two edges with two alternative edges by reversing the intermediate subsequence. For a symmetric distance example, if the old edges are (a, b) and (c, d) and the proposed edges are (a, c) and (b, d), the change is accepted when the new route score is lower. In RouteCraft, the comparison uses the route evaluation logic rather than assuming that every reversal is beneficial. This is a local-search heuristic and may stop at a local optimum.

Chapter 4

Step 4 – Testing and Test Data

4.1 Testing Approach

The native DSA test program `RouteCraftDsaTests.cpp` executes seven test groups: custom-container behavior, route metrics, capacity and customer coverage using Nearest Neighbor, capacity and customer coverage using Clarke–Wright Savings, elastic fleet tiers, strongly connected component filtering, and invalid-input handling. A Java/JNI integration test file, `RouteOptimizerBridgeTest.java`, is also included in the project. The report records the test coverage present in the supplied source; test outcomes should be reproduced on the target machine using the provided build instructions before submission.

4.2 Native Test Cases

The seven groups below are invoked by the native test executable. They describe the assertions present in the supplied source code; execution status must be confirmed on the submission machine.

ID	Test group	Expected behavior checked
T1	Custom containers	Checks dynamic-array insertion and erasure, hash-map lookup and rehashing, duplicate-key handling, and binary-heap ordering.
T2	Route metrics	Checks round-trip distance, time, cost, weighted score and geographic distance helper against known values.
T3	Capacity and coverage – Nearest Neighbor	Checks route capacity, exactly-once customer coverage and route count using Nearest Neighbor.
T4	Capacity and coverage – Clarke–Wright	Runs the same feasibility and coverage checks using the savings construction method.
T5	Elastic fleet tiers	Checks the order of dedicated, ad-hoc and virtual route tiers when the configured fleet is insufficient.
T6	Strongly connected component	Checks that the largest mutually reachable component is retained and edges outside it are removed.
T7	Input validation	Checks that optimization with customers but no depot is rejected with an invalid-argument error.

4.3 Representative Test Data

The native tests use small deterministic scenarios so that expected outcomes can be checked directly. The custom-container test inserts and erases values in a dynamic array, inserts 40 key-value pairs into a hash map to exercise rehashing, and checks min-heap ordering. The route-metric test uses a depot at $(0, 0)$ and a customer at $(3, 4)$ in the test coordinate system. The capacity tests use two depots and three customers with demands 2, 3 and 1 units, with vehicle capacity 5. The elastic-fleet test uses one depot with one dedicated and one ad-hoc vehicle, each of capacity 5, and three customers with demand 5 each; the third route is therefore marked virtual. The SCC test uses two mutually connected nodes and one isolated node. The invalid-input test supplies a customer but no depot.

4.4 Testing Limitations

The supplied native tests verify selected deterministic behaviors. They do not establish optimality, performance on every city-scale OSM extract, or correctness under every possible input. Integration behavior should also be checked on the intended Windows/JDK/compiler setup. Any final claim that all tests passed should be based on a fresh test execution and its recorded output.

Chapter 5

Step 5 – Contributions, Key Takeaways and Future Scope

5.1 Team Contributions

The following division records the team's agreed allocation of the major implementation and documentation responsibilities. Shared integration and review were carried out collaboratively.

Team member	Individual contribution
Matta Jay Paul (106125067)	Route improvement and evaluation module: implementation of 2-opt local search, route metric evaluation and supporting route-improvement functionality.
Mayank Gupta (106125069)	Road-network algorithms: Dijkstra shortest-path processing, Kosaraju strongly connected components and road-graph processing. Also responsible for preparing DSA test cases, test documentation and the project report.
Meesala Hariteja (106125071)	Route construction and optimization coordination: Nearest Neighbor, Clarke–Wright Savings, capacity-constrained route construction, customer clustering and reassignment. Also responsible for the Java application, JNI adapter, Java–C++ communication and overall system integration.

The allocation above should be retained only to the extent that it reflects the work actually performed by each member. Shared work, code review and joint debugging should be acknowledged where applicable.

5.2 Key Takeaways

1. A real routing problem requires more than shortest-path computation; assignment, capacity, fleet availability and route ordering must be considered together.
2. Graph representations and shortest-path algorithms provide the road-network foundation for route planning.
3. Greedy construction heuristics can produce practical candidate routes, while local search

can improve their ordering.

4. Resource constraints can change customer assignment decisions; a geographically nearest depot is not always the best choice when fleet capacity is considered.

5. Modular design and Java–C++ integration require consistent data models, clear interfaces and testing across component boundaries.

5.3 Future Scope

Possible extensions include live traffic updates, delivery time windows, improved global-search or metaheuristic optimization, more advanced multi-depot assignment strategies, electric-vehicle range and charging constraints, richer map-matching, and larger-scale performance evaluation. These are proposed extensions and are not claimed as completed features of the current implementation.

5.4 Acknowledgements

The team thanks the Department of Computer Science and Engineering, NIT Tiruchirappalli, and the CSLR31 course instructor for the opportunity to apply data structures and algorithms to a practical software project. The team also acknowledges the open mapping community and the maintainers of the software tools and libraries used during development.

5.5 References

1. G. Clarke and J. W. Wright, "Scheduling of Vehicles from a Central Depot to a Number of Delivery Points," *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964.

2. E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

3. J. A. Bondy and U. S. R. Murty, *Graph Theory with Applications*. Macmillan, 1976.

4. OpenStreetMap contributors, "OpenStreetMap," https://www.openstreetmap.org/ (accessed 2026).

5. Oracle, "Java Native Interface Specification," https://docs.oracle.com/en/java/javase/ (accessed 2026).

6. CSLR31 Data Structures Laboratory – Project Assignment, Department of Computer Science and Engineering, NIT Tiruchirappalli, 2026.